Valura

Build a multi-agent market intelligence platform on Agno.

Format. Solo build · 11 phases

Audience. Engineers learning the full Agno stack end-to-end.

Outcome. A deployed, multi-user agent platform that exercises every core feature in the Agno framework.

Read this first. The point of this assignment is not to ship a production trading system. The point is to wire every Agno primitive into one coherent product so the framework becomes second nature. Treat acceptance criteria as the unit test. If a phase clicks faster, move on.

Contents

1. Project overview 3

2. Learning outcomes 4

3. Prerequisites and setup 5

4. Phases 1 – 11 in detail 6

5. Feature coverage matrix 16

6. Submission requirements 18

7. Evaluation rubric 19

8. Stretch goals 20

9. References 21

A. Appendix — Valura domain brief 22

B. Appendix — Starter skeleton 23

1. Project overview

What you are building. A multi-user platform called **Valura** that helps an analyst, fund manager, or self-directed investor run daily market research, track a portfolio and watchlist, and propose trades that humans review before they execute. Valura is built as one Agno-powered *AgentOS* service that exposes itself through Slack, Telegram, and a browser UI, persists everything in Postgres, and runs a deterministic morning briefing for every user on a cron.

Why this domain. Money on the line gives every Agno feature an honest reason to exist. RAG is required because models do not know this morning's 10-K. Multi-agent coordination is required because no one model is good at search *and* quantitative analysis *and* news *and* risk. Human-in-the-loop is required because a single hallucinated trade is a real loss. Evals are required because you can grade calls against what actually happened. Tracing is required because when a recommendation is wrong, you need to know which tool returned what.

What 'done' looks like. A single Git repository, deployable to Railway with one script, that runs an Agno AgentOS service with: a Research Team (5 specialized agents), a Morning Briefing Workflow, a PgVector-backed Knowledge base populated with at least three real 10-Ks, per-user memory and watchlists, two tiers of HITL on the trade-proposal tool, input guardrails on user-pasted text, OpenTelemetry tracing into Postgres, four eval suites that you can rerun on demand, and at least two live chat interfaces (Slack + AG-UI). A 10-minute Loom demo and a 1-page learnings memo close out the assignment.

What this is not. A real trading system, a research product you can sell, or anything that should manage actual money. Valura is a training simulator for Agno. Use paper data, sandboxed brokerage endpoints, and fake admin approvers throughout.

2. Learning outcomes

By the end of this assignment, an engineer should be able to, with no additional reading:

- Build an Agno **Agent** from scratch with tools, instructions, streaming, and async tool calls — and explain the run loop.
- Design an Agno **Team**, pick the right **TeamMode** (coordinate / route / broadcast / tasks), and reason about when a team beats a single agent.
- Compose a **Workflow** using **Step**, **Loop**, **Parallel**, **Condition**, and **Router**, and decide when to choose a workflow over a team.
- Wire up a **Knowledge** base on PgVector with hybrid search, ingest PDFs and Markdown, and run both agentic and traditional RAG.
- Configure **memory** (automatic vs. agentic), avoid the *user_id* pitfall, and keep memory operations cheap with a dedicated *MemoryManager*.
- Use the **RunContext** to inject session state, dependencies, and knowledge filters into tools at runtime.
- Implement the three tiers of **human-in-the-loop** (requires_confirmation, @approval, requires_user_input) and explain when each is appropriate.
- Apply **guardrails** (PII, prompt-injection, OpenAI moderation) as pre_hooks and write a custom guardrail.
- Wrap any Agent into **AgentOS**, serve it as a FastAPI app, and connect it to the Control Plane and at least two chat interfaces.
- Stand up **OpenTelemetry tracing** into Postgres, split trace_db from the primary DB, and read traces with SQL.
- Write and run four kinds of **evals** (accuracy, reliability, performance, agent-as-judge) and use their output to fix a real bug.
- Deploy to Railway with pgvector, set RUNTIME_ENV=prd with JWT auth, and pass the production checklist.

3. Prerequisites and setup

Tooling

Install these before starting. None of them are optional.

- Python 3.12+ and uv (or poetry).
- Docker Desktop — you will run Postgres + pgvector locally as agnohq/pgvector:18.
- A Postgres client (psql, TablePlus, or DBeaver) for inspecting agno_sessions, agno_traces, and friends.
- An OpenAI *and* an Anthropic API key. You will swap between them and need fallback configured.
- A free Slack workspace where you can install a bot, a Telegram bot token from @BotFather, and a Railway account.
- Git, plus a personal repo for the project (private is fine).

Recommended reading order

Skim, do not read end-to-end. Touch the page when its phase starts.

- docs.agno.com/introduction — framework framing.
- docs.agno.com/first-agent — the 20-line agent.
- docs.agno.com/agents/overview — building agents.
- docs.agno.com/teams/overview — teams and TeamMode.
- docs.agno.com/workflows/overview — workflows.
- docs.agno.com/knowledge/overview — knowledge and RAG.
- docs.agno.com/memory/best-practices — read before Phase 2.
- docs.agno.com/runtime/human-approval — the three tiers.
- docs.agno.com/agent-os/introduction — the runtime layer.
- docs.agno.com/runtime/observability — tracing strategy.
- docs.agno.com/runtime/deploy — the deploy checklist.

Repo layout (suggested)

```
valura/
  app/
    agents/        # research, data, news, risk, reporter
    teams/         # research_team.py
    workflows/     # morning_briefing.py
    tools/         # custom tools (portfolio_db, trade_proposal, ...)
    knowledge/     # ingestion scripts
```

```
  hooks/         # guardrails, audit_log, post-hooks
  server.py      # AgentOS entrypoint
evals/
  accuracy/
  reliability/
  performance/
  judge/
scripts/
  seed_knowledge.py
  railway_up.sh
docs/
  learnings.md
  postmortem.md
docker-compose.yml
pyproject.toml
README.md
```

4. Phases 1 – 11 in detail

Phase 1 — One agent, one tool

Goal. Build the smallest agent that does useful research. No persistence yet. The point is the run loop.

Tasks

1. Create a single ResearchAgent with YFinanceTools and one web search toolkit (ExaTools or TavilyTools).
2. Give it instructions that force a three-part output: business, financials, risks.
3. Run it with print_response, then switch to arun and stream the output.
4. Make at least two tool calls happen in parallel under aprint_response and prove it from the timing.

Deliverable

A agents/research.py with a working main() block that answers ‘What should I know about NVDA?’ in under 30 seconds.

Acceptance criteria

- ■ Agent returns structured three-part output for any ticker.
- ■ Async run streams tokens before tool calls complete.
- ■ Two tool calls observed running concurrently in the logs.
- ■ No db wired yet — prove statelessness.

Docs to read

- agents/overview, agents/building-agents
- tools/overview, tools/toolkits/data/yfinance

Phase 2 — Persistence: db, sessions, memory

Goal. Turn the stateless agent into a stateful one with proper isolation.

Tasks

1. Stand up Postgres locally via Docker Compose using agnohq/pgvector:18.
2. Wire PostgresDb into the agent and enable add_history_to_context=True with num_history_runs=5.
3. Run a 5-turn conversation, then inspect agno_sessions in psql.
4. Enable update_memory_on_run=True and run the same conversation as two distinct user_id values; verify memories do not bleed.

5. Add a session_state with a watchlist list, template it into instructions ("Watchlist: {watchlist}"), and let a tool mutate it via run_context.session_state.

Deliverable

A second module app/db.py that exposes a configured PostgresDb singleton, plus updated agent that survives a restart with full memory and history.

Acceptance criteria

- ■ After restart, the agent recalls a fact from a prior session.
- ■ Two user_id values produce disjoint memory rows.
- ■ A tool mutation of session_state is visible in the next turn's prompt.
- ■ agno_sessions and agno_memories tables have non-empty rows.

Docs to read

- sessions/overview, sessions/persisting-sessions
- memory/overview, memory/best-practices
- state/agent/overview

Phase 3 — Knowledge and RAG

Goal. Give the agent domain grounding it cannot get from training data, and let it write back what it learns.

Tasks

1. Create a Knowledge backed by PgVector with search_type="hybrid" on the same Postgres instance.
2. Write scripts/seed_knowledge.py that ingests at least three real 10-K PDFs, two earnings-call transcripts, and a handful of your own Markdown notes.
3. Enable search_knowledge=True on the agent (agentic RAG), then run a question that requires hopping between two filings.
4. Add a custom save_insight(title, body, tags) tool that calls knowledge.insert(...). Run a follow-up where the agent uses its own past insight.
5. Add metadata filters: {ticker: "NVDA", year: 2024}. Verify filters are honored.

Deliverable

A populated agno_knowledge table, plus a notebook or script showing a question that the model cannot answer without the RAG hit and clearly can with it.

Acceptance criteria

- ■ Ingestion completes for >= 5 documents.
- ■ Hybrid search returns BM25 + vector hits visibly.
- ■ Agent saves at least one insight back into the KB during a run.

- [ ] Metadata filter reduces result set as expected.

Docs to read

- knowledge/overview, knowledge/quickstart
- knowledge/concepts/* (readers, chunkers, embedders)
- knowledge/vector-stores/pgvector

Phase 4 — Research team with five agents

Goal. Decompose the one big agent into specialists and pick the right coordination mode.

Tasks

1. Create five members: Sourcer (web), DataAnalyst (YFinance + a Postgres tool over your portfolio table), NewsAgent (a news API), RiskAgent (quantitative checks against watchlist), Reporter (writes final answer).
2. Build *three* teams, not one: a **coordinate** team for open-ended research, a **route** team for routing a user message to one specialist, and a **broadcast** team for running every member's risk check on the portfolio.
3. Enable reasoning=True on the coordinate team and capture full reasoning into a debug log.
4. Use a **callable factory** for members so the team composition varies by user role (admin sees RiskAgent; viewer does not).

Deliverable

A teams/ directory with three modules, plus a brief README explaining which mode you chose for which use case and why.

Acceptance criteria

- [ ] All three team modes are exercised end-to-end.
- [ ] Coordinate team produces reasoning trace visible in output.
- [ ] Callable factory returns different members for different roles across two consecutive runs.
- [ ] Sub-agents communicate without shared state leakage.

Docs to read

- teams/overview, teams/delegation, teams/building-teams
- reasoning/overview, reasoning/usage/agents/team-cot

Phase 5 — Deterministic morning-briefing workflow

Goal. Build the daily pipeline. This is the part of Valura that should be reproducible run over run.

Tasks

1. Compose a Workflow with the structure: Parallel(fetch_portfolio, fetch_watchlist_news, fetch_macro) → research_team → Condition(any_red_flag, true=RiskAgent) → Reporter.
2. Wrap the per-ticker analysis in a Loop so the workflow handles any portfolio size.
3. Pass typed step inputs/outputs (StepInput / StepOutput) and access previous-step output explicitly in the Reporter step.
4. Add an early-stop branch: if the macro signal is circuit-breaker, abort the workflow and emit only the macro section.
5. Expose the workflow as a callable tool for the agent in Phase 1 (workflow-as-tool).

Deliverable

workflows/morning_briefing.py plus a runner that produces a Markdown briefing for a sample user with three positions and four watchlist tickers.

Acceptance criteria

- ■ Workflow runs end-to-end against your local DB.
- ■ Parallel block reduces total wall time vs. sequential by >= 30%.
- ■ Conditional branch triggers correctly on a fabricated red flag.
- ■ Workflow callable from the research agent as a tool.
- ■ Re-running with identical input produces functionally identical output (deterministic).

Docs to read

- workflows/overview, workflows/building-workflows
- concepts/workflows/workflow-patterns/*
- workflows/access-previous-steps, workflows/early-stop, workflows/workflow-tools

---

Phase 6 — Multimodal: PDFs, audio, charts

Goal. Use Agno's media primitives to do work the text-only path cannot.

Tasks

1. Ingest 10-K PDFs through the agent itself (not just the seed script) using a tool that accepts a file path and routes it through Agno's file media parameter.
2. Transcribe a short earnings-call audio clip with the appropriate transcription toolkit and feed the transcript into Knowledge.
3. Add a generate_price_chart(ticker, range) tool that returns a ToolResult with images=[Image(...)] so the chart flows back into the model.
4. Build a debugging tool that accepts a screenshot of a competitor dashboard, asks the model to describe it, and saves the description into Knowledge.

Deliverable

Four working media tools and an example run that uses all four in a single conversation.

Acceptance criteria

- ■ PDF tool ingests a real filing in one run.
- ■ Audio transcription produces text searchable in the KB.
- ■ Generated chart flows through ToolResult.images back to the model.
- ■ Screenshot description is stored and retrievable.

Docs to read

- multimodal/overview, multimodal/agent/*
- tools/overview (built-in parameters: images, audio, files)

Phase 7 — HITL, guardrails, hooks

Goal. Make the system safe for a user whose mistakes cost real money.

Tasks

1. Mark propose_trade(ticker, side, qty) with @tool(requires_confirmation=True). Pause a run, approve via continue_run, then reject and verify the tool was not executed.
2. Mark propose_large_trade (notional > $10K) with @approval(type="required") and a second tool with type="audit". Stand up a fake ‘admin’ approver loop.
3. Add pre_hooks=[PIIDetectionGuardrail(mask_pii=True), PromptInjectionGuardrail()] to the user-facing agent and write a test that fails closed when a known injection payload is sent.
4. Write one **custom guardrail**: block any input containing a ticker the user is not permitted to trade (per session_state).
5. Add a post_hook with run_in_background=True that writes to your own audit_log table.

Deliverable

Updated trade tool, populated agno_approvals table, an audit_log table you defined yourself, and a short SECURITY.md explaining the three tiers.

Acceptance criteria

- ■ Trade is blocked on rejection, executed on confirmation, auto-routed to admin above threshold.
- ■ PII payload is masked before it reaches the model.
- ■ Prompt-injection payload is rejected with a structured error.
- ■ audit_log table has rows for every approved action.
- ■ Background post-hook does not block the user-facing response time.

Docs to read

- runtime/human-approval

- guardrails/overview, guardrails/included/*
- hooks/overview, hooks/post-hooks

Phase 8 — AgentOS, Slack, AG-UI

Goal. Turn the bag of agents into a service real users can talk to.

Tasks

1. Wrap everything in AgentOS(agents=[...], teams=[...], workflows=[...], db=db, tracing=True) and run fastapi dev.
2. Open os.agno.com, connect to your local OS, chat, inspect a trace tree, and approve a HITL request from the UI.
3. Register the **Slack** interface against a real bot, wire it to the route team, and use resolve_user_identity=True.
4. Register the **AG-UI** interface and confirm the same agent answers correctly from both Slack and the browser, with memory following the user across surfaces.
5. Add one direct FastAPI route (/webhooks/news) that invokes the news agent on incoming payloads.

Deliverable

A running AgentOS exposed on localhost:8000 with at least two interfaces wired and a documented webhook.

Acceptance criteria

- ■ AgentOS UI shows live traces with tool inputs and outputs.
- ■ Slack thread reuses the same session across multiple replies.
- ■ Same user_id in Slack and AG-UI shares memory.
- ■ Custom webhook successfully triggers a run.

Docs to read

- agent-os/introduction, agent-os/run-your-os
- runtime/serve-as-api, runtime/interfaces
- agent-os/interfaces/slack/*, agent-os/interfaces/ag-ui/*

Phase 9 — Scheduler and background execution

Goal. Make the briefing run by itself, and let slow jobs not block users.

Tasks

1. Use the AgentOS scheduler to fire the morning-briefing workflow at 06:00 in the user's timezone, per user, posting to their Slack DM.

2. Add a long-running backtest(strategy, range) tool. Invoke it with background=True, return a run_id, and poll until completion.

3. Add a cancel_run code path that aborts a running backtest cleanly.

4. Confirm the schedule survives a process restart by inspecting agno_schedules and agno_schedule_runs.

Deliverable

A scheduled job visible in the AgentOS UI, plus a polling client that survives an agent restart mid-job.

Acceptance criteria

- ■ Cron triggers the workflow on time, on consecutive runs.
- ■ Background run returns a run-id within 1 second.
- ■ Cancellation stops the job and persists a cancelled state.
- ■ Schedule re-loads after restart.

Docs to read

- runtime/scheduling, agent-os/scheduler/overview
- runtime/serve-as-api (background execution section)
- advanced/run-cancellation

Phase 10 — Four eval suites

Goal. Make quality measurable. You cannot improve Valura without this.

Tasks

1. **Accuracy.** Write 20 input/output gold-standard pairs for research questions where you know the right answer (historical events with public outcomes). Score with LLM-as-judge.

2. **Reliability.** Specify that the morning briefing must call fetch_portfolio exactly once and RiskAgent if any position dropped >5%. Assert.

3. **Performance.** Set a latency budget per agent (Reporter < 4s, RiskAgent < 2s) and fail the suite on regression.

4. **Agent-as-Judge.** Grade Reporter outputs against three custom criteria: *evidence-cited*, *balanced*, *actionable*. Save grades into agno_evals.

5. Use the output of one eval to fix a real bug — ship the fix with a note in docs/learnings.md.

Deliverable

Four eval modules under evals/, a single make eval target, and a report committed to docs/eval_report.md.

Acceptance criteria

- ■ All four suites run with one command.

- ■ At least one eval suite catches a regression you intentionally introduce.
- ■ Eval results persisted to agno_evals.
- ■ A documented fix traceable to an eval failure.

Docs to read

- evals/overview, evals/introduction
- concepts/evals/* (accuracy, reliability, performance)
- evals/agent-as-judge/overview

Phase 11 — Deploy, demo, memo

Goal. Ship it. Then explain it.

Tasks

1. Deploy AgentOS to Railway via scripts/railway_up.sh with managed Postgres + pgvector.
2. Set RUNTIME_ENV=prd and JWT_VERIFICATION_KEY. Confirm the runtime requires JWTs on every endpoint.
3. Split db and trace_db into two Postgres instances. Confirm traces no longer write to the primary.
4. Walk the production checklist: HTTPS, health check, secrets in the manager, persistent DB volume, tracing on, guardrails on, HITL on irreversible tools.
5. Record a 10-minute Loom demo and write a 1-page docs/postmortem.md covering: what you would change, what surprised you, what Agno made easy, what it made hard.

Deliverable

A live URL, the Loom demo, and the postmortem committed to the repository root.

Acceptance criteria

- ■ /health returns 200 from the public hostname.
- ■ JWT auth rejects unauthenticated calls.
- ■ Slack interface works against the deployed URL.
- ■ trace_db rows growing, primary db not growing in the spans table.
- ■ Loom + postmortem submitted.

Docs to read

- runtime/deploy, agent-os/usage/database-migrations
- runtime/security-and-auth
- runtime/observability (multi-db tracing)

---

5. Feature coverage matrix

Use this as the self-check at the end of each phase. If a feature has not been touched by the end of its row, that phase is not done.

1	Agent class, model selection, tools, instructions, run / arun, streaming, concurrent async tool calls.
2	PostgresDb, sessions and runs, history-in-context, automatic memory, user_id isolation, session_state, RunContext.
3	Knowledge, readers (PDF/MD), chunkers, embedders, PgVector, hybrid search, agentic RAG, self-learning loop, metadata filters.
4	Team, all four TeamMode values, delegation, sub-teams, callable-factory members, reasoning on teams, cache_callables.
5	Workflow, Step, Loop, Parallel, Condition, Router, StepInput/StepOutput, access-previous-steps, early stopping, workflow-as-tool.
6	Multimodal media parameters (images, audio, files), ToolResult with images, transcription, document understanding.
7	requires_confirmation, @approval(type=...), requires_user_input, PII guardrail, prompt-injection guardrail, custom BaseGuardrail, pre/post hooks, run_in_background.
8	AgentOS, FastAPI app, Control Plane, OpenTelemetry tracing, Slack interface, AG-UI interface, custom FastAPI routes, session-per-surface.
9	Scheduler (cron), background execution, run cancellation, agno_schedules persistence.
10	Accuracy evals, reliability evals, performance evals, agent-as-judge evals, agno_evals table.
11	Deploy to Railway, JWT-RBAC, production checklist, multi-DB tracing (db + trace_db), HTTPS, secrets manager.

Features explicitly *not* required by this assignment but encouraged in stretch work: multi-framework wrappers (ClaudeAgent, LangGraphAgent, DSPyAgent), MCP tools, the Workspace tool for local file workflows, conversational workflows (beta), context compression (beta), and custom interfaces.

6. Submission requirements

Everything below lands in the same Git repository.

- **Source code.** The full valura/ repo with the directory layout from Section 3, runnable via docker compose up locally.
- **README.md.** One screen long. What it is, how to run it, how to deploy it. Nothing else.
- **docs/learnings.md.** One bullet per phase summarizing the single thing you would tell yourself before starting.
- **docs/postmortem.md.** One page. What would you change in Agno, what would you change in your build, what would you keep.
- **docs/eval_report.md.** Auto-generated from your eval runs. Latest scores per suite, plus one fixed bug traced to an eval failure.
- **SECURITY.md.** The HITL tiers and which tools fall into each.
- **Loom demo (~10 min).** Walk through a Slack conversation, a morning briefing run, a HITL approval, and one eval suite running.
- **Live URL.** A Railway deployment that survives one full day of inactivity and still serves /health.

7. Evaluation rubric

Scored 0 (missing) / 1 (partial) / 2 (meets bar) / 3 (excellent) per row. Total out of 30.

Agent & tool design	10%	Tools have clean schemas, docstrings, type hints. Async parallelism is visible in traces.
Team & workflow composition	15%	TeamMode choices are justified. Workflow is deterministic, parallelism gives a real speedup, conditional branches are exercised.
Knowledge & memory	10%	Hybrid search clearly beats vector-only on at least one query. Memory respects user_id isolation under load.
HITL & guardrails	15%	All three HITL tiers used appropriately. Custom guardrail demonstrates the BaseGuardrail pattern. Audit table is real, not a stub.
AgentOS & interfaces	10%	Two interfaces share memory through the same user_id. Control plane is reachable, traces render.
Evals	15%	Four suites green. At least one bug is documented as found-by-eval and fixed.
Deploy & ops	10%	JWT enforced in prod. trace_db separated. Health check passes. Secrets in a manager, not in source.
Code quality & docs	10%	Readable code. README explains the system in 5 minutes. Postmortem is honest and specific.
Demo & memo	5%	Loom is concise. Memo names two non-obvious learnings.

Passing bar. 18/30 overall, with no individual row scoring 0. Below that, schedule a 30-minute pairing session and target the weakest two rows.

8. Stretch goals

Reach for these only after every acceptance box on every phase is checked. Each one adds depth on a feature area the base assignment only grazes.

- **Multi-framework.** Replace the NewsAgent with a ClaudeAgent wrapper around a Claude Agent SDK agent, or a LangGraphAgent wrapper around a LangGraph graph. Confirm AgentOS serves it identically.
- **MCP integration.** Connect MCPTools to a real or mock brokerage MCP server and route trade execution through it.
- **Workspace tool.** Add Workspace("./notes", allowed=["read", "list", "search"], confirm=["write", "edit"]) and let the agent maintain its own research notes on disk.
- **Custom interface.** Subclass BaseInterface and build a Discord-channel or email-inbox interface that the framework does not ship.
- **Conversational workflow.** Convert the morning briefing into a conversational workflow so users can ask follow-ups against the same execution context.
- **Context compression (beta).** Turn it on for long-running research sessions and measure the token savings against a fixed workload.
- **Multi-tenant.** Add a real tenant boundary at the JWT scope level so two organizations can run on the same deployment without seeing each other’s data.

9. References

Open these alongside your editor, not in advance. The Agno docs are structured so each page corresponds to a phase here.

- Home and SDK introduction: docs.agno.com and docs.agno.com/introduction
- First agent: docs.agno.com/first-agent
- Agents: docs.agno.com/agents/overview, /agents/building-agents, /agents/running-agents
- Teams: /teams/overview, /teams/building-teams, /teams/delegation
- Workflows: /workflows/overview, /workflows/building-workflows, /concepts/workflows/workflow-patterns/overview
- Tools and toolkits: /tools/overview, /tools/toolkits/overview
- Models: /models/overview, /models/providers/model-index
- Knowledge: /knowledge/overview, /knowledge/concepts/*, /knowledge/vector-stores/pgvector
- Memory: /memory/overview, /memory/best-practices
- Sessions, state, context: /sessions/overview, /state/agent/overview, /context/overview
- Multimodal: /multimodal/overview, /multimodal/agent/*
- Reasoning: /reasoning/overview, /reasoning/usage/agents/*
- Guardrails and HITL: /guardrails/overview, /runtime/human-approval
- Hooks: /hooks/overview
- AgentOS: /agent-os/introduction, /agent-os/run-your-os, /agent-os/interfaces/*
- Runtime: /runtime/overview, /runtime/serve-as-api, /runtime/storage, /runtime/observability, /runtime/security-and-auth, /runtime/interfaces, /runtime/scheduling, /runtime/deploy
- Evals: /evals/overview, /concepts/evals/*, /evals/agent-as-judge/overview
- Reference (API): /reference/agents/agent, /reference/teams/team, /reference/workflows/workflow
- Cursor rules and coding agents: /other/cursor-rules, /coding-agents

Appendix A — Valura domain brief

Who uses Valura. An analyst, a small fund's junior PM, or a serious retail investor who wants a research assistant that runs in the background, talks to them in Slack, and never executes a trade without an explicit click.

What Valura does in a day. At 06:00, the morning briefing workflow runs per user: pulls the user's portfolio and watchlist out of Postgres, fetches overnight news, runs a risk pass against position concentration, and posts a Markdown briefing into the user's Slack DM. Throughout the day, users ask ad-hoc questions in Slack or AG-UI — 'What did NVDA management say about export controls in the last call?', 'Show me anyone in my portfolio with >15% revenue concentration to China', 'Draft a trade proposal: sell 100 NVDA.' Trade proposals are paused for the user's confirmation; over $10K notional they additionally route to the PM (admin) before anything posts to the brokerage MCP endpoint.

What Valura knows. A Knowledge base seeded with 10-Ks, transcripts, and the user's own notes. Memory tracks the user's stated risk preferences, watchlist changes, and which kinds of recommendations they tend to act on. session_state holds the active 'view' (portfolio, watchlist, macro-focus) and travels with the conversation.

Sample queries to test against.

- "What should I know about NVDA before earnings on Wednesday?"
- "Run a concentration check on my current portfolio."
- "Summarize what each of my watchlist companies said about AI spend in their last call."
- "Propose a trade: sell half my AAPL position."
- "Save a note: management quality at TSMC is the under-discussed moat."
- "What did I tell you last month about my risk tolerance?"

Hard constraints. No real money. No real brokerage. Use a stub execute_trade that writes to a Postgres trades table. Every trade tool is gated by HITL.

Appendix B — Starter skeleton

Drop this into app/server.py as the starting point and grow it phase by phase. Do not start by writing all five agents; let them appear when the assignment asks for them.

```
# app/server.py
from agno.agent import Agent
from agno.db.postgres import PostgresDb
from agno.models.anthropic import Claude
from agno.os import AgentOS
from agno.tools.yfinance import YFinanceTools
from agno.tools.exa import ExaTools

DB_URL = "postgresql://ai:ai@localhost:5532/ai"
db = PostgresDb(db_url=DB_URL)

research = Agent(
    name="Research",
    model=Claude(id="claude-sonnet-4-5"),
    db=db,
    tools=[YFinanceTools(), ExaTools()],
    instructions=[
        "You are Valura, a market research assistant.",
        "For any ticker question, return three sections: "
        "Business, Financials, Risks.",
        "Always cite sources you actually used.",
    ],
    add_history_to_context=True,
    num_history_runs=5,
    update_memory_on_run=True,
    markdown=True,
)

agent_os = AgentOS(
    agents=[research],
    db=db,
    tracing=True,
)
app = agent_os.get_app()

if __name__ == "__main__":
    agent_os.serve(app="app.server:app", reload=True)
```

Docker Compose to keep it local

```
# docker-compose.yml
```

```
services:
  agentos:
    build: .
    ports: ["8000:8000"]
    env_file: .env
    depends_on: [db]
  db:
    image: agnohq/pgvector:18
    environment:
      POSTGRES_USER: ai
      POSTGRES_PASSWORD: ai
      POSTGRES_DB: ai
    ports: ["5532:5432"]
    volumes:
      - pgdata:/var/lib/postgresql/data
volumes:
  pgdata:
```

Minimal pyproject dependencies

```
# pyproject.toml (excerpt)
[project]
name = "valura"
requires-python = ">=3.12"
dependencies = [
    "agno[os]",
    "anthropic",
    "openai",
    "psycopg[binary]",
    "pgvector",
    "fastapi",
    "uvicorn[standard]",
    "yfinance",
    "exa-py",
    "pypdf",
    "pdfplumber",
]
```

Start small. The shortest path is: clone empty repo, drop the skeleton above, run `docker compose up db -d`, `uv pip install -e .`, `fastapi dev app/server.py`, open os.agno.com, connect to localhost:8000, ask a question. If that round-trip works, the rest of the build is downhill.